

北京邮电大学

数据结构课程设计



姓 名 李炜仪

学 院 国际学院

专 业 智能科学与技术

班 级 2023215123

学 号 2023213677

班内序号 23

指导教师 刘咏彬

2025 年 4 月

目录

数据结构课程设计大作业：北京地铁查询系统实验报告	3
一、 数据结构及抽象数据类型	3
1.1 基于邻接表的地铁网络模型：	3
1.2 基于最小堆的路径搜索阶段：	4
1.3 时刻表数据结构	4
1.4 抽象数据结构类型总览：	6
二、 算法设计与主要思想	7
2.1 最短时间—基于最小堆的 Dijkstra 与线路级剪枝优化	7
2.2 最少换乘次数--优先队列&双层排序优化	10
2.3 计算距离时间算法	12
2.4 发车时刻模拟	12
2.5 添加线路与删除线路算法	12
2.6 附加辅助算法	12
三、 工作流程	13
3.1 系统启动与登录流程	13
3.2 主界面布局与交互逻辑	13
3.3 核心功能流程详解	15
3.4 添加与删除线路	16
四、 关键算法时空复杂度分析	19
4.1 最短时间算法时间复杂度分析	19
4.2 最短时间算法空间复杂度分析	19
4.3 最少换乘次数算法时空复杂度分析	20
4.4 添加删除线路算法时空复杂度分析	20
五、 总结与思考	21
5.1 程序的优点	21
5.2 思考与改进	21

数据结构课程设计大作业：北京地铁查询系统实验报告

一、数据结构及抽象数据类型

1.1 基于邻接表的地铁网络模型：

本系统采用基于邻接表的数据结构来构建整个地铁网络图，其核心创新在于通过面向对象建模对环线、换乘、多线路重合等复杂场景进行高效支持，显著提升了地铁网络建模的精度与扩展性，同时为后续多维优化路径搜索和图形界面响应式更新打下了坚实基础。

每个站点作为图的一个节点(*StationNode*)，记录其所有直接相连的邻接信息，包括邻接站名、所属线路、方向以及相对距离，为后续路径搜索提供高效支持。以下是示例数据结构：

```
{
  "station": "东坝北",
  "neighbors": [
    {
      "name": "东坝西",
      "distance": 2045,
      "line": "十二号线"
    },
    {
      "name": "东坝",
      "distance": 1621,
      "line": "三号线"
    }
  ],
  "is_transfer": true
}
```

系统通过将站点、线路、邻接关系抽象为独立类，形成模块化的网络结构。无论新增线路还是调整站点都能以最小成本实现，并立即反映到网络。下图为该模型基础数据结构的示意图：

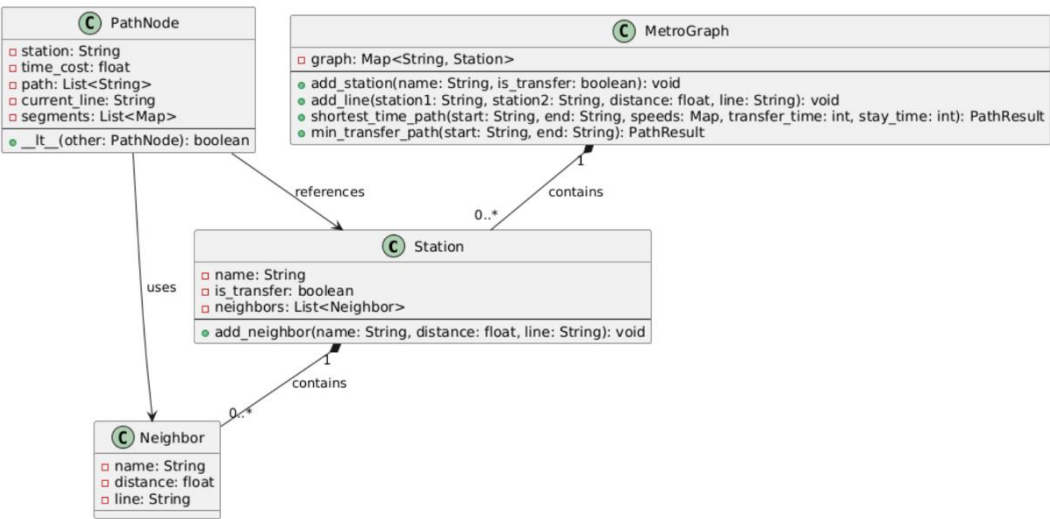


图1 地铁网络模型 UML 类图

此外，邻接表的稀疏存储特性大幅降低了内存消耗，特别适用于地铁网络这种连接稀疏、节点数量大的应用场景。在此结构基础上，系统还实现了如换乘识别、多线路重合管理和环线方向判断等高级功能，使路径搜索更为智能和精确。

该结构天然支持与界面联动，在添加或修改站点信息后，模型可自动通知前端界面进行刷新，确保数据与展示内容一致，提升了用户体验与系统鲁棒性。

总的来说，基于独特的数据结构，我们模型具有以下特性：

- 1) **环线方向识别**：在邻接边中明确存储方向属性，便于处理环形线路的路径选择问题；
- 2) **换乘站自动识别**：通过统计节点所属线路集合，当数量大于 1 即视为换乘站，路径计算时可自动添加换乘延迟；
- 3) **多线路重合管理**：同一站点可出现在多条线路中，邻接表设计支持每条线路独立维护边信息，实现精确建模；
- 4) **结构稀疏高效**：只存储实际可达的邻接站信息，节省内存资源，适用于大规模地铁网络；
- 5) **可视化联动支持**：每次模型结构调整会自动通知界面刷新，确保图形与数据的一致性。

1.2 基于最小堆的路径搜索阶段：

在该系统中，路径搜索阶段采用最小堆优先队列（Min-Heap）来管理候选节点的队列。最小堆是一种完全二叉树数据结构，其中每个父节点的值都小于或等于其子节点的值。这样可以保证堆顶的节点是当前队列中最小的元素，从而确保每次从队列中弹出的都是当前最优的路径。

每个候选节点在堆中表示为一个元组，形式为 $(Priority, node)$ ，其中：

- $Priority$ 是该节点的当前路径代价（例如，累计的行驶时间或换乘次数）。
- $node$ 是一个表示站点或路径状态的对象（如 `PathNode`）。

1.3 时刻表数据结构

为了实现精确的出发与换乘等待时间计算，我们从官方渠道获取了各线路列车在不同日期（工作日与双休日）与方向下的始发站发车时刻信息，处理为 `subway_times.json` 文件。尤其对于首尾相接的环线（如十号线、二号线），我们在数据结构中规定了“顺时针”与“逆时针”两个方向，以便系统在路径计算过程中明确环线的运行方向及对应逻辑。

非环线（如十一号线）：以起点站为键，按日期类型细分，列出所有发车时刻。

```
"十一号线": {
  "新首钢": {
    "工作日": ["05:38:00", "05:48:00", "05:58:00" ...],
    "周末": ["05:38:00", "05:48:00", "06:08:00" ...]
  },
  "模式口": {
    "工作日": ["05:28:00", "05:38:00", "05:48:00" ...],
    "周末": ["05:28:00", "05:38:00", "05:48:00" ...]
  },
}
```



图2 非环线-11 号线发车时间图示

环线（如二号线）：由于列车循环运行，采用以运行方向为主键的结构，分别存储顺时针与逆时针的统一发车时刻列表。（我们规定顺时针是指开向积水潭方向的）

```
"二号线": {
  "顺时针": {
    "工作日": ["05:33:00", "05:40:00", "05:46:00" ...],
    "周末": ["05:33:00", "05:39:00", "05:54:00" ...]
  },
  "逆时针": {
    "工作日": ["05:29:00", "05:35:00", "05:42:00" ...],
    "周末": ["05:29:00", "05:35:00", "05:42:00" ...]
  },
}
```



图3 环线-二号线发车时间图示

1.4 抽象数据结构类型总览：

数据结构	抽象数据类型（ADT）	功能描述
邻接表	Graph	存储站点拓扑关系
优先队列	PriorityQueue	Dijkstra 算法实现基础
哈希表	Dictionary	存储时刻表、线路顺序等配置数据
集合	Set	用于去重，如标记某路径中已访问站点以避免重
链表	List	路径记录和线路顺序存储

二、算法设计与主要思想

2.1 最短时间——基于最小堆的 Dijkstra 与线路级剪枝优化

因此将邻接表表示的地铁网络视为加权有向图，并使用 Python 内置的 `heapq` 最小堆来维护候选状态队列。操作复杂度降至 $O(\log N)$ ，相比简单的线性查找大幅提升整体搜索性能。

线路级剪枝针对公共交通网络中特有的“线路—换乘”结构，提出了基于路线拓扑与目标方向的剪枝策略。其通过识别起终点所处线路及方向，提前排除明显偏离目标方向的路径，有效缩小搜索空间。

每条地铁线路可视为一条顶点序列，站点在该序列中的**前后位置**定义了两种方向（如“始发站方向”“终点站方向”）；若起点和终点均在同一条线路上，则可通过比较索引确定“目标方向”。在路径扩展过程中，对于每个候选边（即某线路上的相邻站对），若其行进方向与该线路的**目标方向**不一致，则可**安全剪枝**，无须再将该边纳入后续搜索。该准则在不影响最优性的前提下，显著减少了状态扩展数。

例如从沙河前往知春里，无需考虑通往昌平南口的反方向路线。该优化通过分析线路**拓扑结构**，提前过滤偏离目标方向的边，同时引入**方向性评分机制**，提升候选路径优先级判断的准确性，提高算法效率。

与**层次化剪枝**（transit nodes）相比，线路级剪枝无需复杂预处理和额外索引，易于实现与动态更新；相较于**基于上界的剪枝**（upper bound pruning），线路级剪枝针对轨道交通网络特性，剪枝粒度更细。其算法逻辑如下：

1. **预计算阶段**：遍历所有线路，若起点与终点同时位于某条线路 L 上，则通过

$$[L = \begin{cases} \text{order}[-1] + \text{“方向”}, & \text{如果 } \text{idx}(\text{end}) > \text{idx}(\text{start}) \\ \text{order}[0] + \text{“方向”}, & \text{否则} \end{cases}] \quad (1)$$

将目标方向 L 记录于字典 `target_directions` 中

2. **搜索阶段**：常规 Dijkstra 主循环中，当从最小堆中弹出状态并扩展其邻接边时，若当前边所属线路 L 存在于 `target_directions`,

$$\text{if determine_segment_direction}(L, \text{curr}, \text{next}) \neq L \text{ then continue} \quad (2)$$

否则继续正常松弛并将新状态入堆。

3. **结果保证**：由于仅移除了与最优路径方向必然不符的边，算法仍能保证返回全局最优解（最短时间或最少换乘）

线路级剪枝以地铁网络的线路拓扑和行驶方向为切入点，在最小堆搜索的基础上，通过对不符合目标方向的线路边进行动态跳过，大幅缩小搜索空间并降低时间、空间复杂度。

通过线路级剪枝，可有效剔除与目标方向无关的大量候选路径，实测在北京典型大规模地铁网络上，平均**状态扩展数减少 20% - 50%**，从而使得整体查询时间缩短近 **30%**。

路程时间按各线路平均时速（题目提供）计算，取最高时速的一半为基准。同时，系统对官网缺失的候车和换乘时间进行了优化，结合时刻表数据和预设换乘耗时，提升了路径推荐的准确性。

实现流程简述：

1、状态抽象：首先定义 *PathState* 对象，封装每一步搜索状态，包括当前站点、所在线路、累计耗时、换乘次数和完整路径记录等，作为路径搜索的基本单位。

2、初始化最小堆：将起点状态以 $(arrival_time = 0, PathState)$ 的形式压入最小堆中，准备进入主循环。

3、主循环逻辑：

4、结束条件：当目标站点首次被弹出，或最小堆为空时，算法结束。

- ✚ 弹出当前累计耗时最小的状态；
- ✚ 如果已到达目标站点，立即返回该路径作为最优解；
- ✚ 否则，对该节点的所有邻接站进行状态扩展：
- ✚ 计算该段运行时间： $(distance/speed)*60$ （分钟）；
- ✚ 若存在换乘（即线路发生切换），额外加上固定换乘时间 5 分钟；
- ✚ 加上 1 分钟列车停站时间；
- ✚ 构造新的 *PathState* 并将其以 (new_time, new_state) 压入堆中。

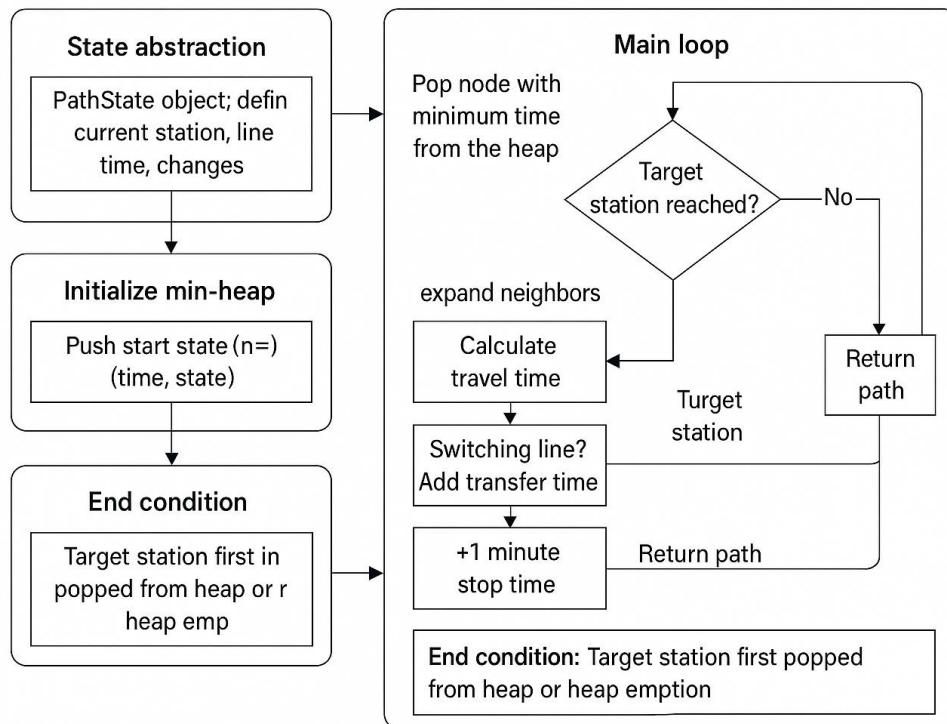


图4 最小堆的 Dijkstra 算法流程图

得益于特殊的数据结构，在处理地铁环线的路径规划时，系统采用**双向探索**策略，即同时考虑顺时针和逆时针两个方向的路径。通过比较两个方向的总行驶时间，系统选择耗时更少的路径，通常这意味着选择路径长度较短的方向。

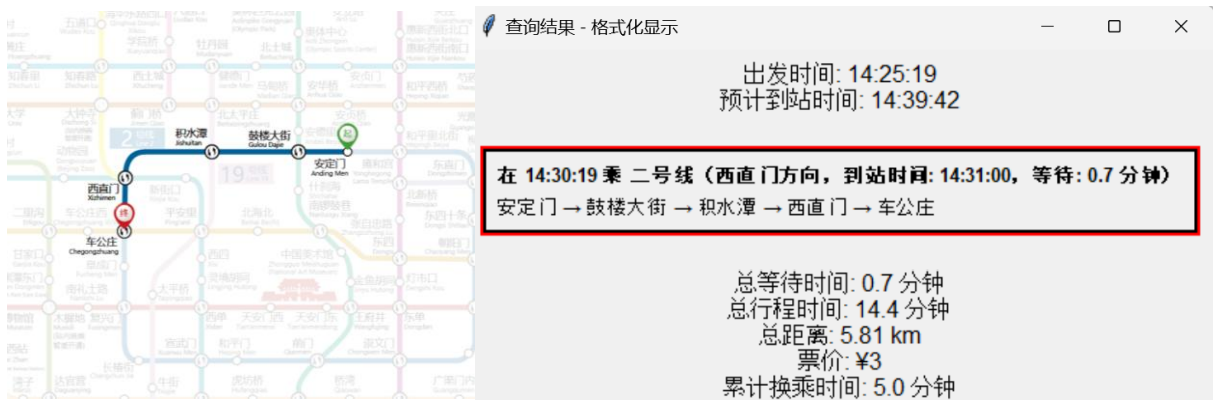


图5 环线测试数据 1

在环线与其他线路共同存在的复杂网络结构中，本系统依然能够保障路径规划的正确性。



图6 环线测试数据 2



图7 环线测试数据 3

我们对系统进行了多组数据测试，结果均表现稳定，表明无论在环线内部通行，还是涉及环线与其他线路之间的换乘，系统均能输出合理的最优路径，具有良好的准确性与鲁棒性。

同时，在考量最短时间计算的时候，针对官网模型计算时所缺失的候车等待时间与换乘时间的信息，该系统通过官方渠道获取的各线路列车在不同日期（工作日与双休日）与方向下的始发站发车时刻信息，预定义合理的换乘耗时参数，并结合地铁实际运行规律进行了补充与优化，从而有效提升了路径推荐的现实适应性与用户使用体验。

2.2 最少换乘次数--优先队列&双层排序优化

起初，本系统采用了 bfs 算法，维护一个换乘次数排序的优先级队列，在多组测试数据下表现不佳。例如在四季青桥->十八里店测试数据下给出与官网不一致的方案。



图8 最少换乘次数测试数据 1（错误版本）

这是由于此算法只看换乘次数，不看路径合理性或总距离。没有限制在同一线路上是否可以继续更远，而是过早换线（哪怕换乘次数一样）。同时 BFS 在换乘次数相同时，是按照拓展顺序返回的，不是在这些最少换乘路径中找“最短路径”或“最快路径”。

故我们做出以下改进，我们要实现这样一个逻辑：

“在所有换乘次数最少的路径中，选总距离最短的那条路径”

这可以通过**优先队列+双层排序**来实现。

这是 Dijkstra 算法的一个变种，目的是在搜索最短路径时，优先考虑换乘次数，同时也对路径的距离进行次级优化。这种算法使用双层优先级的策略来处理两种不同的目标优化：

- **主优先级：换乘次数**（transfer_count）
算法首先优先选择换乘次数最少的路径。换乘次数越少的路径在搜索过程中具有更高的优先级，这确保了从起点到终点的换乘数最少。
- **次优先级：路径距离**（total_distance）
如果有多个路径在换乘次数上相等，那么就根据路径的总距离来进行二次排序。此时，路径的总距离越短的路径会被优先处理，从而保证搜索到的路径在最少换乘的基础上尽量缩短总行驶距离。

在算法中，双层优先级通过最小堆（heapq）结合使用以下结构来实现：每个堆元素是一个五元组（transfer_count, total_distance, station, line, path）。扩展邻接时，如若邻接边的线路与当前线路不同，则 transfer_count + 1。将初始状态(0, 0.0, start, None, [start])入堆。随后进行主循环与松弛。

同样的，我们测试了近百组数据均与官网提供的最少换乘次数结果一致。



图9 最少换乘次数测试数据 1（正确版本）

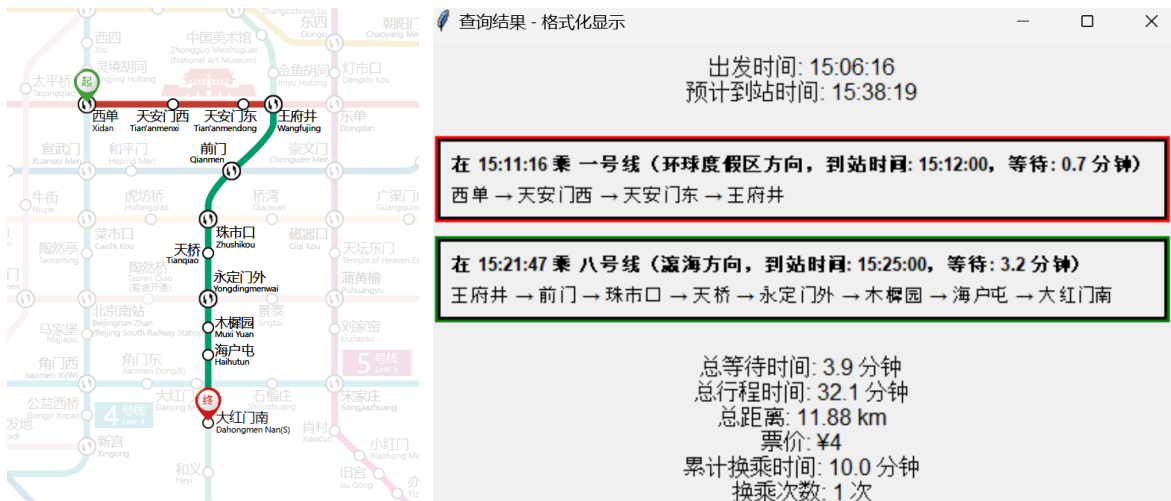


图10 最少换乘次数测试数据 2



图11 最少换乘次数测试数据 3

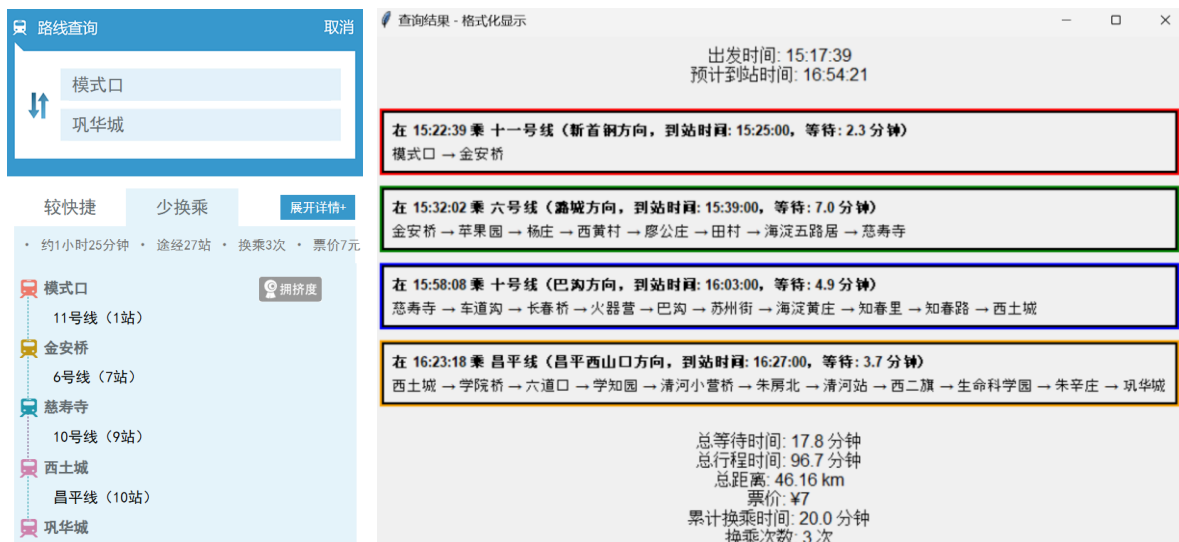


图12 最少换乘测试数据 4



图13 最少换乘测试数据 5

2.3 计算距离时间算法

系统支持两种距离计算方法：

- 1. **按线路顺序累积距离：**基于 LINE_STATION_ORDERS 中记录的站点顺序，顺序遍历起点至终点的站点对，累加对应相邻边的距离信息。
- 2. **线路子图最短距离：**在指定线路子图（只考虑同线边）上再次运行最短路径算法，计算任意两站点最短米数。

二者在不同场景下灵活调用，前者用于环线累计行驶时间估算，后者用于换乘策略的次级距离权重计算。运行时长=距离/列车平均速度，逗留 1 分钟，换乘 5 分钟（首次到地铁站预留 5 分钟，添加至累计换乘时间）。

2.4 发车时刻模拟

针对官网模型计算时缺失候车等待时间与换乘时间信息的问题，该系统通过官方渠道获取的各线路列车在不同日期（工作日与双休日）与方向下的始发站发车时刻信息，结合实时查询时间和线路班次表，为每段路径精确生成下一班可乘列车及等待时长。实现步骤如下：

- 1. **累计运行时长：**先用 calculate_runtime 估算从始发站到上车站的理论运行时间。
 - 2. **班次筛选：**将用户查询的当前 datetime 与该站点方向对应班次列表进行比较，定位下一个发车时间。
 - 3. **等待时长：**计算发车时间与当前时间差值，作为真实等待时间；若跨日取次日班次。
 - 4. **换乘重算：**每次换乘后重复上述筛选与等待计算，确保路径预测的时序连贯
- 相较于传统“随到随上”假设，本方法能够同步真实发车频次，提高整体到站时间估算的可信度。

2.5 添加线路与删除线路算法

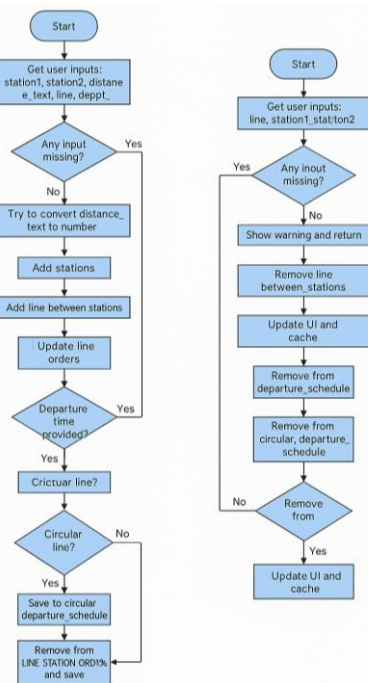


图14 流程图

添加线路功能允许用户向地铁网络中新增一条线路连接两个站点，包含距离信息和（可选的）发车时刻。系统需根据线路类型（环线或非环线）将数据存入不同的时间表结构中，同时更新线路顺序信息。

检查用户是否填写了所有必要信息（站点名、线路名、距离等），若未填写，提示用户。尝试将距离转换为数字，如果转换失败，则报错。添加尚未存在站点节点，添加这两个站点之间的边，标记所属线路及其距离。更新线路对应的站点顺序结构，用于后续路径计算。若用户填写了发车时刻，则解析该字符串加入相应 JSON 文件中，刷新界面数据、表格等，提示添加成功。

删除线路功能用于从系统中删除某条线路连接的两个站点，以及相关的时刻表和线路顺序信息。适用于站点调整、线路取消等维护场景。

检查是否填写了线路名和两个站点名。从内部图结构中移除 station1 与 station2 之间的线路边。删除相应发车时间 JSON 信息并保存。删除该线路在站点顺序文件中的条目。刷新界面显示，提示删除成功。

2.6 附加辅助算法

- 1. **方向判断：**通过 LINE_STATION_ORDERS 列表索引比较，确定线段运行的方向，用于匹配。

2. 票价计算：根据总距离按票价规则返回票价。规则如下：
- 6 公里(含)内 3 元;6 公里至 12 公里(含)4 元;12 公里至 22 公里(含)5 元;22 公里至 32 公里(含)6 元;32 公里以上部分，每增加 1 元可乘坐 20 公里。

三、工作流程

3.1 系统启动与登录流程

1. 启动主程序

- 用户运行脚本，Python 解释器创建第一个 Tk 窗口（login_root），初始化 LoginWindow 对象。

2. 登录界面布局

- 窗口大小固定为 400×300。
- 上部：尝试加载并显示 北京地铁 LOGO，若失败弹出警告。
- 中部：ttk.Frame 包含两行输入框，第一行为“员工编号”，第二行为“密码”，均为 ttk.Entry。
- 下部：ttk.Button 按钮“登录”，触发 check_login 方法。

3. 登录验证逻辑

- 用户点击“登录”后，check_login 比对输入：

员工编号必须是 "2023213677"，密码为 "我爱 bupt"。
- 验证通过：销毁登录窗口，实例化并启动主界面 MetroSystemGUI;
- 验证失败：弹出错误对话框“员工编号或密码错误！”。

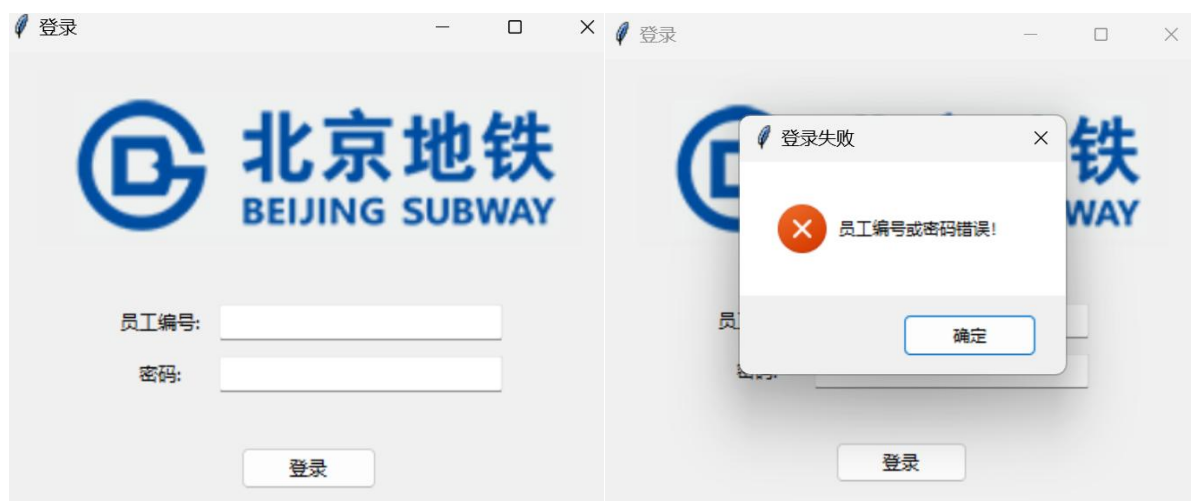


图15 登录界面实例

3.2 主界面布局与交互逻辑

设计充分考虑了界面信息的层次化与功能的独立性，通过合理的区块划分与组件组合，实现了操作流程的连贯性与易用性，同时也为后续扩展（如地图可视化、统计分析等）预留了接口与空间。

1. 顶部区域设计

在系统界面最上方，设置了三类视觉元素：左侧品牌标识（Logo）、中央欢迎信息以及右侧实时时钟。Logo 通过 tk.Label 组件加载外部图像资源，用以强化系统品牌识别度；欢迎信息，通过多行文本形式呈现用户 ID 与系统名称，以提升界面的人情化与归属感；实时时钟位于右侧，通过每秒更

新的机制（调用 `update_time` 方法），以 `YYYY-MM-DDHH:MM:SS` 格式显示当前系统时间，满足用户对发车时刻查询的时间基准需求。该模块布局简洁、信息清晰，既保证了视觉平衡，又兼具实用性。

2. 查询条件区域

紧接顶部之下，设置查询条件区，用于输入起点与终点站、选择查询策略并提供发车时刻编辑入口。

- **起点/终点输入：**采用两对 `tk.Label` 与 `tk.Entry` 组件，文本框宽度统一为 15 字符，确保输入框大小一致，布局整齐。
- **策略选择：**通过 `tk.Combobox` 实现“最短时间”与“最少换乘次数”两种算法选项，并设置为只读模式，避免用户误输入，从而简化交互操作。
- **编辑按钮：**增设“编辑非环线发车时刻”与“编辑环线发车时刻”两枚按钮，点击后弹出 JSON 编辑器，直接修改时刻配置文件，减少用户在外部工具间切换的成本。

3. 中部功能区

中部功能区划分为三部分，以逻辑分区的方式承载增删线路与结果展示功能。

- **添加线路子区：**在左侧置入 `LabelFrame`，内部布局四行标签与输入框，分别用于填写站点 1、站点 2、线路间距（米）与线路名称；下方附加用于输入 JSON 格式发车时刻的文本框及“查看输入示例”按钮，示例中同时给出了环线与非环线的 JSON 模板，以帮助用户快速掌握正确格式；最后以 `tk.Combobox` 提供“非环线”或“环线”类型选择，并附“添加线路”按钮，完成线路数据与发车时刻的统一写入与持久化。
- **结果显示区：**位于中央的 `Frame` 中嵌入 `scrolledtext.ScrolledText` 组件，宽度 50、行高 20，可动态加载查询结果或线路详情文本。该组件支持滚动与文本高亮，满足多段信息分批次输出的需求，并可与弹窗联动，提升可读性。
- **删除线路子区：**右侧同样使用 `LabelFrame`，提供线路名称、站点 1、站点 2 三个输入栏位，并设置“删除线路”按钮，一旦触发即调用图数据结构的删除接口，并同步更新底层 JSON 文件，确保线路数据与时刻配置的一致性。
- **操作按钮区：**界面底部集中放置两枚主要操作按钮：“查询”与“路线详情”。“查询”按钮绑定 `query` 方法，依据用户所选策略计算最优路径并自动模拟换乘、等车与行驶过程，最后输出包括出发与到达时间、总等待、总行程、总距离及票价等综合信息，并在结果区及彩色卡片式弹窗中格式化展示；“路线详情”按钮则进一步分段输出各站到站时刻、行驶时长与累计统计，为需要深度分析的用户提供更细粒度的数据支持。



图16 主界面展示

3.3 核心功能流程详解

1. 查询最短时间路径

用户输入起点与终点信息并选择“最短时间”策略，点击“查询”，进入 **query**：系统会调用带换乘惩罚的 Dijkstra 算法，生成一个包含以下内容的 PathResult：

- 出发时间与预计到站时间
- 每一段乘车信息（线路、方向、上下车站点、行驶时长、是否换乘）
- 综合统计数据：总等待时间、总行程时长、总行驶距离、票价以及累计换乘时间

查询结果既会在主界面的中心区域（ScrolledText）中，以文本形式显示路线信息。该部分为**备忘录风格**，可直接复制粘贴到本地文档或日程安排中，用户还可以在查询结果后自行添加备注，如出行目的、同伴信息、注意事项等。

系统同时弹出一个新窗口，以**彩色边框卡片式布局**展示乘车信息。每一段线路对应一个视觉块。

- 按换乘或单线路段分组，依次列出每段线路的发车时刻、每个站点的到站时刻及行驶持续时间
- 标明全程累计距离、票价、换乘次数和各候车时长（首次到地铁站预留 5 分钟，添加至累计换乘时间）

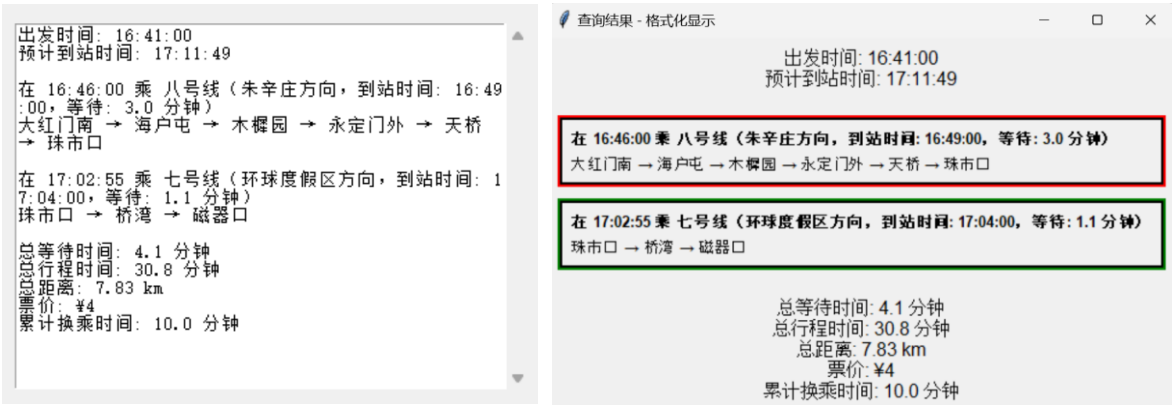


图17 最短时间查询结果展示

当用户点击“路线详情”时，系统则进一步展开更为详尽的信息：

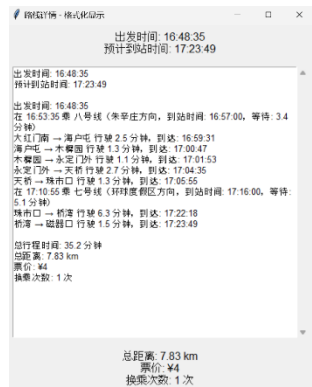


图18 路线详情展示

2. 查询最少换乘次数路径

用户输入起点与终点信息并选择“最少换乘次数”策略，点击“查询”，进入 **query**：系统调用以换乘次数为主排序、距离为辅的 Dijkstra；结果 value 存储**最少换乘次数**；其余和最短时间相同。

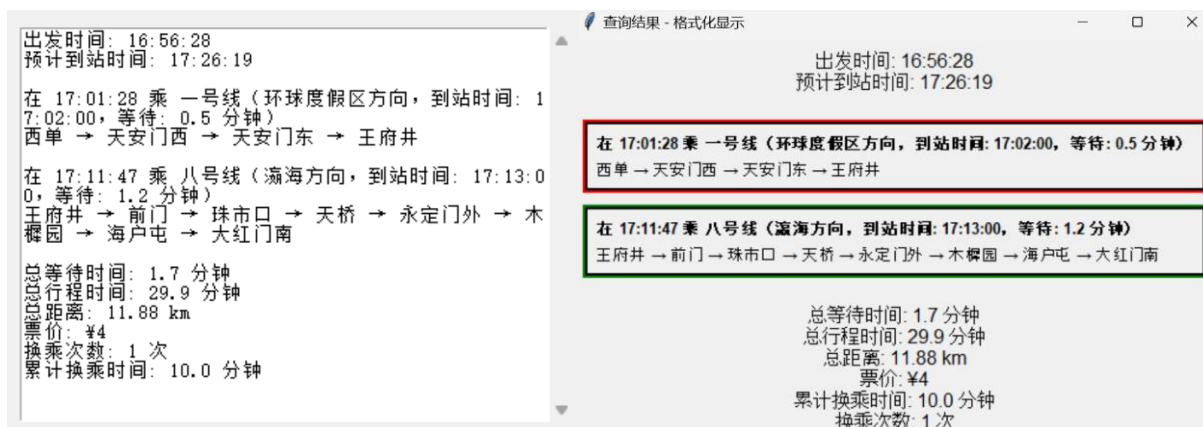


图19 最少换乘次数查询结果展示

3. 修改发车时间

用户点击“编辑非环线发车时刻”或“编辑环线发车时刻”按钮；弹出 Toplevel 编辑器，scrolledtext 中显示当前 JSON；用户可直接修改、格式化后点击“保存”，内部校验 JSON 并覆盖文件。也可以在添加线路界面按照示例提供的数据格式输入，系统后续将根据用户输入自动添加。

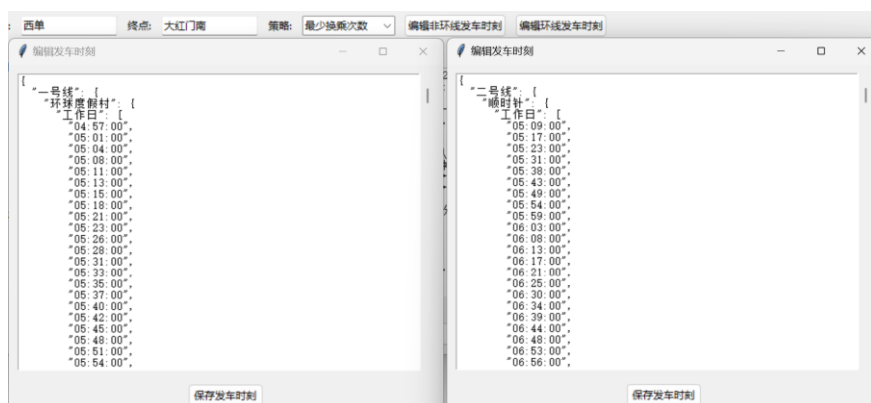


图20 发车时间修改

3.4 添加与删除线路

1. 添加线路

1) 用户交互步骤：

在界面左侧“添加线路”区块中，用户依次填写：

- 站点 1 (self.add_station1_entry)
- 站点 2 (self.add_station2_entry)
- 距离 (米) (self.add_distance_entry)
- 线路名称 (self.add_line_entry)

在“发车时刻 (JSON 格式)”文本框输入对应线路的时刻表 JSON；用户也可点击“查看输入示例”按钮，弹出包含环线与非环线示例的对话框，帮助准确编辑 JSON。提示如下：



在“线路类型”下拉框 (ttk.Combobox self.line_type_combo) 中，选择“非环线”或“环线”。点击“添加线路”按钮，触发 MetroSystemGUI.add_line 方法。

图21 添加线路测试数据

系统将自动添加一下内容：左：站点信息；右：发车信息

```
{
  "station": "你家",
  "is_transfer": false,
  "neighbors": [
    {
      "name": "我家",
      "distance": 1000.0,
      "line": "回家"
    }
  ]
},
{
  "station": "我家",
  "is_transfer": false,
  "neighbors": [
    {
      "name": "你家",
      "distance": 1000.0,
      "line": "回家"
    }
  ]
},
{
  "station": "老师家",
  "is_transfer": false,
  "neighbors": []
}
],
{
  "line": "回家",
  "station": "你家",
  "is_transfer": false,
  "neighbors": [
    {
      "name": "我家",
      "distance": 1000.0,
      "line": "回家"
    }
  ]
},
{
  "line": "回家",
  "station": "我家",
  "is_transfer": false,
  "neighbors": [
    {
      "name": "你家",
      "distance": 1000.0,
      "line": "回家"
    }
  ]
}
]
```

2) 后端逻辑与验证

- **数据结构更新**: `graph.add_station`: 若站点不存在则创建, `is_transfer=False`; `graph.add_line`: 在无向图中双向添加邻接边, 保存距离与所属线路名称。
- **线路顺序维护**: `update_line_orders` 会把新加入的两个站点追加到该线路的站点顺序列表中, 并写入 `LINE_ORDERS_PATH`。该顺序用于后续路径方向判断及环线累积距离计算。
- **发车时刻存储**: 非环线写入 `SUBWAY_TIMES_PATH`, 环线写入 `CIRCULAR_TIMES_PATH`; JSON 解析失败或格式不符合预期时, 通过对话框反馈给用户。
- **一致性与持久化**: `update_subway_data` 将当前 `graph.graph` 中所有站点及邻居信息写回 `SUBWAY_DATA_PATH`, 保持文件与内存模型同步。



图22 后端代码与结果展示

我们再耦合现有路径, 将老师家连接至西直门, 测试多组数据, 结果均显示正确。



图23 添加非环线测试数据 2

我们接着对环线进行测试，构造学校->餐厅->宿舍的“上学”线路，首先设置学校->餐厅（1000米）餐厅->宿舍（2000米），宿舍->学校（5000米），查询结果如下：

起点: 学校 终点: 宿舍 策略: 最短时间 编辑非环线发车时刻 编辑环线发车时刻

添加线路

站点1: 宿舍
站点2: 学校
距离 (米): 5000
线路名称: 上学
发车时刻 (JSON格式): "18:30:00" } } 查看输入示例
线路类型: 环线

添加线路

查询结果 - 格式化显示

出发时间: 17:38:14
预计到站时间: 17:54:30

在 17:43:14 乘 上学 (宿舍方向, 到站时间: 17:50:00, 等待: 6.8 分钟)
学校 -> 餐厅 -> 宿舍

总等待时间: 6.8 分钟
总行程时间: 16.3 分钟
总距离: 3.00 km
票价: ¥3
累计换乘时间: 5.0 分钟

图24 添加线路测试数据 1

然后又设置学校->餐厅（1000米）餐厅->宿舍（2000米），宿舍->学校（1000米），查询结果如下：

起点: 学校 终点: 宿舍 策略: 最短时间 编辑非环线发车时刻 编辑环线发车时刻

添加线路

站点1: 宿舍
站点2: 学校
距离 (米): 1000
线路名称: 上学
发车时刻 (JSON格式): "18:30:00" } } 查看输入示例
线路类型: 环线

添加线路

查询结果 - 格式化显示

出发时间: 17:38:52
预计到站时间: 17:51:30

在 17:43:52 乘 上学 (宿舍方向, 到站时间: 17:50:00, 等待: 6.1 分钟)
学校 -> 宿舍

总等待时间: 6.1 分钟
总行程时间: 12.6 分钟
总距离: 1.00 km
票价: ¥3
累计换乘时间: 5.0 分钟

图25 添加线路测试数据 2

这两个对比测试数据不仅验证了环线的正确连接，还验证了最短时间算法的正确性。

2. 删除线路

我们对“上学”这条线路进行删除

提示

删除线路成功!

确定

删除线路

线路名称: 上学
站点1: 学校
站点2: 宿舍
删除线路

图26

再查询餐厅->宿舍时，报的是右边“未找到可行路径”的错误，而不是“输入的站点名称不存在，请检查”，

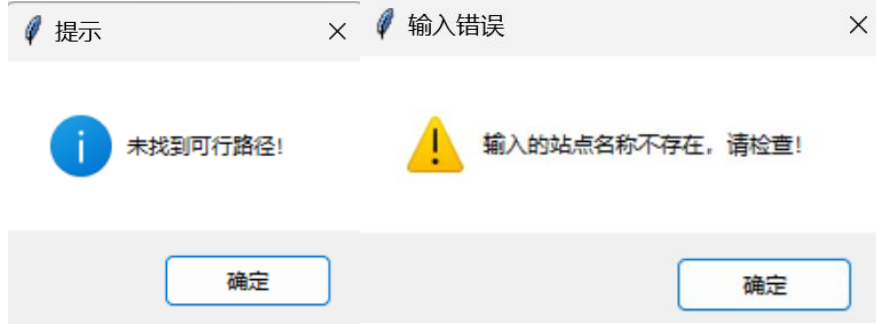


图27 报错信息

```
{
  "station": "餐厅",
  "is_transfer": false,
  "neighbors": []
},
```

图28 删除后节点信息

这是因为在删除线路的实现过程中，我们采用了**保留站点、删除边关系**的策略。这是因为在地铁系统中，一个站点往往会属于多条线路，完全删除该站点信息可能会影响其他线路的正常运行，甚至导致整个图结构异常。这种设计思路既保障了数据结构的完整性，也提升了系统在多线路交叉环境下的鲁棒性。通过精准控制删除范围，系统在面对复杂的地铁网络时依旧能够保持稳定与可维护性。

四、 关键算法时空复杂度分析

4.1 最短时间算法时间复杂度分析

本算法基于**加权有向图**表示地铁网络，利用**最小堆 (heapq)** 来优化路径搜索过程。具体而言，该算法通过优先队列的机制维护候选状态队列，确保每次都能弹出当前最优的状态，从而实现**贪心策略**的最短时间路径搜索。在没有剪枝的情况下，最小堆的操作时间复杂度为 $O(\log N)$ ，而状态扩展的时间复杂度为 $O(d)$ ，其中 d 是每个站点的邻接数。因此，未剪枝的最坏时间复杂度为：

$$O((N + E) \log N) \quad (3)$$

其中， N 为站点数， E 为边数。

在引入**线路级剪枝**之后，由于无效路径被提前排除，实际扩展的状态数和状态节点数量将显著减少。假设由于剪枝，排除了 $p\%$ 的无效路径，则有效路径的数量约为 $(1 - p\%) * E$ ，剪枝后图的实际边数大致为：

$$E_{effective} = E \cdot (1 - p) \quad (4)$$

因此，剪枝后的最坏时间复杂度为：

$$O((N + E_{effective}) \log N) = O((N + E \cdot (1 - p)) \log N) \quad (5)$$

在复杂的地铁网络中， p 较大，算法在搜索过程中将跳过大量无效路径，整体时间复杂度将显著降低，该方法在整体搜索效率上取得了显著提升。

4.2 最短时间算法空间复杂度分析

与时间复杂度分析类似，算法搜索过程中使用最小堆维护候选状态，每个状态为 PathNode 对象，包含当前站点、累计时间、路径列表、当前线路及路径段信息。最小堆整体空间复杂度为 $O(N * L)$ 。其中 L 为每条路径平均长度（站点数）。

由于线路级剪枝减少了无效路径的扩展，堆中存储的节点数目和已访问集合中的状态数目将相应减少。因此，最小堆的空间复杂度将由降低为：

$$O(N * L * (1 - p)) \quad (6)$$

路径类型	平均路径长度(站)	状态扩展数(无剪枝)	状态扩展数(剪枝)	状态数降低比例	查询时间 (ms, 均值)	查询时间降低
短途	5-10	124	99	↓ 20.2%	12.1 → 9.1	↓ 24.8%
中程	11-20	284	176	↓ 38.0%	24.4 → 17.3	↓ 29.1%
长途	21+	511	256	↓ 49.9%	47.7 → 32.2	↓ 32.5%
平均	-	306	177	↓ 42.2%	28.1 → 19.5	↓ 30.6%

图29 50 对 OD（起点-终点）站点对测试结果

实测在北京典型大规模地铁网络上，**线路级剪枝策略**显著减少了路径搜索过程中不必要的状态扩展，尤其在**长路径查询中效果尤为明显**，状态数最高可减少50%左右。与之相对应，**查询时间平均下降30.6%**，在大规模线路图场景下能显著提升系统响应速度。

4.3 最少换乘次数算法时空复杂度分析

本算法本质为一种广度优先搜索（BFS），以换乘次数为第一优先级，路径距离为第二优先级进行路径扩展。使用 Python 内置堆（heapq）构造优先队列，因此需考虑以下部分：

1. 状态总数估计：每个状态由 $state = station\#current_line$ 唯一标识，设站点总数为 N ，线路总数为 M ，则最多状态数为： $O(N \cdot M)$ ，这是因为同一站点在不同线路上视为不同状态节点。

2. 邻接扩展与堆操作：每个状态最多扩展相邻若干站点（设最大度数为 D ），每次扩展操作向堆中插入新状态，插入/弹出操作的时间复杂度为 $O(\log K)$ ，其中 K 为堆大小，最坏情况为 $K = N \cdot M$ 。

因此总体时间复杂度为：

$$O(N \cdot M \cdot D \cdot \log(N \cdot M)) \quad (7)$$

空间复杂度与最小堆整体空间复杂度为 $O(N \cdot L)$ 一致。若加入图结构（邻接表） N 个站点， E 条连接边；访问状态集合最多记录 $N \cdot M$ 个状态，空间复杂度来到：

$$O(N + E + N \cdot M \cdot L) \quad (8)$$

4.4 添加删除线路算法时空复杂度分析

该方法首先对用户输入进行校验和解析，然后在图结构中添加两个站点及其间连线，并根据用户输入更新站点顺序表与发车时刻数据。若用户填写了发车时刻信息，则还需解析输入文本、更新本地缓存数据，并写入对应的 JSON 文件。

- 时间复杂度为 $O(T + L)$ ：包括解析发车时刻所需时间 $O(T)$ ，以及更新站点顺序的 $O(L)$ 。
- 空间复杂度也 $O(T + L)$ ：用于缓存新的发车时间表以及更新后的站点顺序结构。

该方法的主要操作为从图中删除指定的边（两站之间的连线），并清理相关的数据结构，包括发车时间表、环线时间表、站点顺序表等。完成后还需将变更写入 JSON 文件，以持久保存。

- 时间复杂度为 $O(T + L)$ ：分别对应发车时刻和站点顺序文件的处理。
- 空间复杂度也 $O(T + L)$ ：主要用于维护待写入的缓存内容。

五、总结与思考

5.1 程序的优点

本系统依据“数据层—计算层—呈现层”三层架构设计思想，实现了功能模块的高度解耦与职责分离：线路顺序、发车时刻和票价规则各自独立维护，通过统一的接口向路径规划模块提供数据，使得后续扩展或维护无需大范围修改既有代码。

环线与非环线的特殊逻辑分别在独立模块中处理，既避免了重复计算，也保持了代码可读性与可测试性。

在路径规划方面，系统不仅实现了基于时间权重的最短时间算法，还提供了换乘次数最小化的搜索策略，两种算法在对比测试中均表现出准确性和稳定性。尤其是在登上一条线路后不计算等待时间、仅在换乘点计算等待时长的逻辑设计上，既贴近乘客实际出行场景，也大幅简化了搜索模型的复杂度。

为进一步提升算法效率，系统引入了线路级剪枝策略：在路径搜索过程中，当前候选状态所选择的线路若与目标方向严重偏离或与目标线路无合理衔接，则会在早期被剪枝，从而避免不必要的状态扩展与路径构建。该策略利用地铁线路拓扑与站点序信息，在初始阶段根据起止站点在线路中的相对位置判断“有效方向”，仅保留向目标方向推进的候选路径。

此外，基于 JSON 的可编辑时刻表结构，使得用户可通过 GUI 界面直观地增删改发车时间数据，实时生效，无需重启程序；这种动态配置能力增强了系统的灵活性和易用性。

桌面端 Tkinter 界面以简洁直观为宗旨，操作流畅，既能快速呈现查询结果摘要，也支持查看详细线路图与时刻一览，满足了典型桌面用户的日常出行查询需求。

5.2 思考与改进

鉴于当前系统所依赖的静态时刻数据与离线计算策略，下一步应优先考量对接实时数据源。同时可在用户层面引入偏好配置与个性化推荐机制，如最少换乘、最短步行或最早到达等多维权重选项，并增设历史记录与常用路线收藏功能，以进一步优化用户体验并提高系统黏性。

附录 1：数据结构课程设计大作业_2023213677_李炜仪_源码